Take-Home Challenge:
Build a provider-agnostic RAG orchestrator

Problem statement

Ultralytics needs to improve the speed and accuracy of LLM responses while keeping costs from rising.

Today, our system retrieves and injects external context too aggressively. Retrieval is currently applied to nearly every request, even when it is unnecessary. This increases:

- Latency
- Token usage
- Response noise
- Infrastructure cost

We currently support two major response types:

1. **Fast responses, like:**
    a. Website chatbot
    b. Interactive product chat
    c. Real-time support interactions

Primary goal: Extremely fast responses while maintaining strong accuracy

2. **Slow responses, like:**
    a. GitHub issue replies
    b. Async technical support
    c. Long-form troubleshooting

Primary goal: Higher reasoning quality and accuracy, even if latency is slightly higher

The current Ultralytics system also has:

- No response review layer
- Limited orchestration intelligence
- Overly aggressive RAG injection
- Insufficient MCP/tool orchestration abstraction
- Heavy provider coupling

We want a new architecture built completely from scratch in a new repository.

The system must be fully vendor-agnostic, and must work cleanly across major providers, including and not excluding:

- OpenAI
- Anthropic
- Gemini
- DeepSeek
- Kimi

Advanced orchestration features such as:

- MCP usage
- Retrieval strategies
- Thinking/reasoning budgets
- Structured tool usage
- Review pipelines

The goal is not to build a polished end-user application. The goal is to demonstrate strong judgment in LLM systems engineering:

- Selective retrieval
- Orchestration quality
- Multi-turn context management
- Provider abstraction
- Latency optimization
- Cost-aware reasoning
- Practical tradeoff decisions

Why this matters

We want to improve performance across four dimensions:

- **Accuracy:** Responses should be more reliable and contextually correct.
- **Speed:** Interactive systems should feel fast and responsive.
- **Cost Efficiency:** The orchestrator should reduce unnecessary:
    - Retrieval
    - Token usage
    - Reasoning overhead
    - Model calls

Architectural flexibility

The system should not depend on one vendor or proprietary workflow. Switching providers should be straightforward.

Requirements

1. Use retrieval selectively

The system must NOT retrieve context for every request. It should:

- Determine whether retrieval is actually necessary
- Avoid retrieval for generic or unrelated questions
- Retrieve only when retrieval meaningfully improves answer quality
- Minimize unnecessary context injection

The challenge specifically evaluates judgment around:

- When retrieval helps
- When retrieval hurts
- How much retrieval is appropriate

Important: Our current system always injects the retrieval context, even when unnecessary. Your system should improve this behavior significantly.

2. Handle multi-turn conversations

The system should:

- Support follow-up questions
- Reuse prior conversational context intelligently
- Determine when a new retrieval is needed
- Avoid duplicate retrieval/context injection across turns
- Avoid repeatedly sending the same documents back to the model

We want to see thoughtful conversation-state management.

3. Support differentiated response modes

Implement at least two orchestration paths:

1. **Fast path, optimized for:**
    a. Low latency
    b. Minimal retrieval
    c. Lightweight reasoning
    d. Interactive UX

Examples:

- Chatbot
- Live assistant
- Quick support questions

2. **Deep reasoning path, optimized for:**
    a. Higher accuracy
    b. Deeper analysis
    c. Richer retrieval/tool usage
    d. Response review/refinement

Examples:

- GitHub issue responses
- Debugging assistance
- Async support workflows

The system should clearly demonstrate:

- How routing decisions are made
- How orchestration differs between modes
- How reasoning budgets differ between modes

4. Be provider agnostic

The architecture must support multiple providers cleanly. It cannot only work with OpenAI. It must support at least:

- One OpenAI-compatible provider
- One non-OpenAI provider

Examples:

- OpenAI
- Anthropic
- Gemini
- DeepSeek
- Kimi

Provider switching should require minimal code/configuration changes.

Provider-specific implementations should be isolated behind a clean abstraction layer.

We want to evaluate:

- Abstraction quality
- Portability
- Extensibility

5. Improve MCP / Tool usage

The system should demonstrate thoughtful MCP/tool orchestration patterns.

We are particularly interested in:

- Selective tool usage
- Dynamic tool routing
- Structured orchestration
- Cross-provider compatibility
- Minimizing unnecessary tool calls

Advanced orchestration concepts should remain portable across providers.

6. Retrieve from a knowledge source only when needed

The system should:

- Query a retrieval layer or RAG database only when beneficial
- Inject minimal focused context
- explain or demonstrate why the specific context was selected
- avoid indiscriminate context stuffing

We strongly prefer: Small targeted context windows over dumping large irrelevant chunks into prompts

7. Add response review/verification

The current Ultralytics system has effectively no review layer.

Your solution should include some form of:

- Answer validation
- Self-review
- Response critique
- Confidence scoring
- Verification pass
- Lightweight evaluator

This does NOT need to be overly complex.

The goal is to demonstrate practical reasoning about improving answer reliability.

8. Clean implementation

Requirements:

- Open-source GitHub repository
- Clear project structure
- Readable code

- Modular architecture
- Extensible design
- Good engineering practices

Deliverables

Submit a GitHub repository containing:

- **Code:** Implementation of the orchestration system.
- **README, which includes**:
  - Setup instructions
  - Architecture overview
  - Assumptions
  - How to run the project
  - Provider abstraction design
  - Orchestration design decisions

A simple Python script that is runnable with some API keys would suffice.

You can use as much AI as you wish.

What we care about most:

- Systems thinking
- Orchestration quality
- Practical engineering judgment
- Latency/cost tradeoffs
- Retrieval quality
- Provider abstraction design
- Maintainability
- Reasoning about real-world production constraints

We care significantly more about thoughtful architecture and orchestration decisions than UI polish or framework complexity.

Bonus: Seat-based tool cost optimization

Bonus points if the system can intelligently offload suitable tasks to seat-based tools such as Claude Code, Codex, or similar fixed-cost environments instead of always using metered API calls.

We are interested in designs that can:

- Identify tasks better suited for seat-based tools
- Reduce token/API spend where possible
- Use coding agents for repo analysis, debugging, refactoring, or implementation tasks
- Preserve provider-agnostic orchestration principles
- Avoid unnecessary LLM API calls when a fixed-cost tool can complete the work

The goal is not to force all work into these tools, but to demonstrate smart cost-aware routing between:

- API-based model calls
- retrieval systems
- MCP/tools
- seat-based coding or reasoning environments

Strong submissions may show how the orchestrator decides when this type of offloading is appropriate and how results are reviewed before being returned to the user.